编程实验 1

1．**题目：**

班上 5 个学生（学号，姓名，数学成绩），有若干同学成绩不及格，采用顺序表编程实现下面的问题：

函数 1：顺序表输入 5 个同学的数据

函数 2：输出表中不及格同学的数据（成绩<60）。

源程序：

实验-1.c 实验-2.c

计算机软件基础-实验-1 (全局范围)

```c
#define _CRT_SECURE_NO_WARNINGS
/*刘超然-2023060904025-题目1*/
#include<stdio.h>
#include<string.h>
#define MAX 5

//定义学生信息结构体
typedef struct {
    int ID;             //学号
    char name[30];      //姓名
    float mathValue;    //数学成绩
}infor;

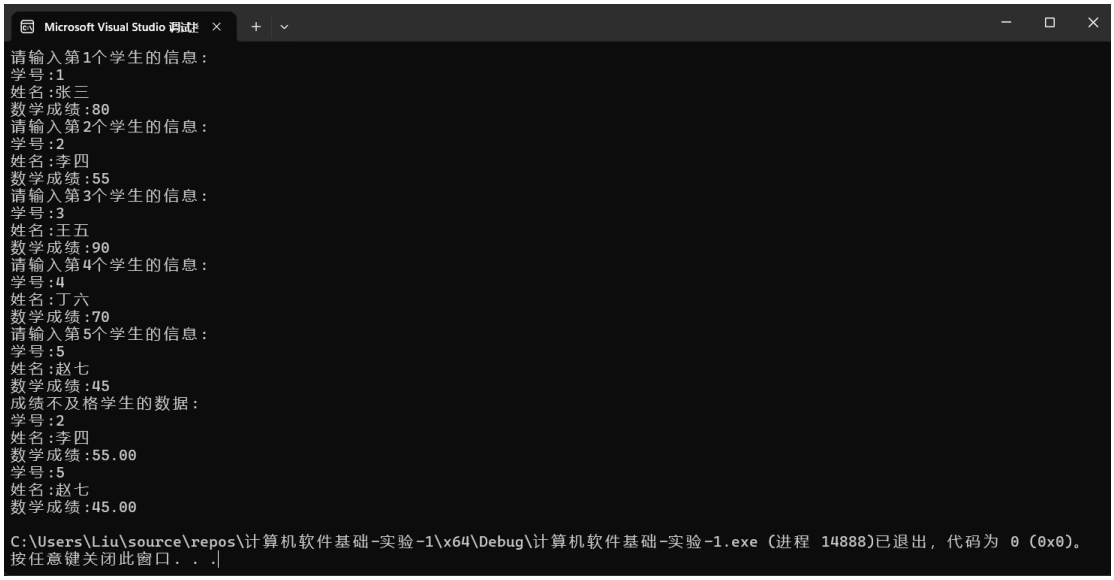
//函数声明
void inputInformation(infor student[], int number);     //输入学生信息的函数
void outputFailStu(infor student[], int number);        //输出不及格学生信息的函数

int main() {
    infor student[MAX];                 //定义顺序表储存学生的信息
    inputInformation(student, MAX);     //输入学生的信息
    printf("成绩不及格学生的数据:\n");
    outputFailStu(student, MAX);        //输出不及格学生的信息
    return 0;
}

//输入学生信息
void inputInformation(infor student[], int number) {
    for (int i = 0; i < number; i++) {
        printf("请输入第%d个学生的信息:\n", i+1);
        printf("学号:");
        scanf("%d", &student[i].ID);
        printf("姓名:");
        scanf("%s", student[i].name);
        printf("数学成绩:");
        scanf("%f", &student[i].mathValue);
    }
}

//输出不及格学生信息
void outputFailStu(infor student[], int number) {
    for (int i = 0; i < number; i++) {
        if (student[i].mathValue < 60) {
            printf("学号:%d\n姓名:%s\n数学成绩:%.2f\n", student[i].ID, student[i].name, student[i].mathValue);
        }
    }
}
```

运行结果：



心得体会：

在处理学生成绩管理的编程过程中，我深刻体会到数据结构的重要性。使用结构体来定义学生的信息使得代码更加清晰，逻辑更加直观。通过顺序表实现学生数据的输入和输出，增强了对数组和循环的理解。同时，考虑到成绩不及格的筛选，我认识到条件判断的灵活应用对于数据处理至关重要。

2．**题目：**

1）已知两个集合 A、B（成员为整数），求两个集合的并集，结果存放于 A 中。

算法思想：依次取 B 中的一个元素与 A 中所有元素比较，不相等则插入到 A 的末尾。

2）已知两个集合 A、B （成员为整数），求两个集合的交集，结果存放于 A 中。

算法思想：依次取 A 中的一个元素与 B 中所有元素比较，不相等则从 A 中删除。

采用顺序表编程实现。要求包含：1）顺序表结构定义、数据初始化、结果输出；2）并/交集函数功能定义；3）主程序调用。

源程序：

```c
#define _CRT_SECURE_NO_WARNINGS
/*刘超然-2023060904025 - 题目2*/
#include <stdio.h>
#define MAX 100

// 定义顺序表结构
typedef struct {
    int data[MAX];       //存放集合元素
    int length;          //当前集合的长度
} set;

// 函数声明
void initList(set* list);            //初始化顺序表的函数
void printList(set list);            //输出顺序表元素的函数
void unionSet(set* A, set B);        //求A和B的并集的函数，将结果存放于A
void intersectSet(set* A, set B);    //求A和B的交集的函数，将结果存放于A

int main() {
    //定义集合 A 和 B
    set A, B;
    //输入 A B 集合元素个数
    int Anumber, Bnumber;
    printf("输入A集合元素个数:");
    scanf("%d", &Anumber);
    printf("输入B集合元素个数:");
    scanf("%d", &Bnumber);

    //初始化集合 A 和 B
    initList(&A);
    initList(&B);

    //令集合长度等于元素个数
    A.length = Anumber;
    B.length = Bnumber;

    //输入集合A的元素
    printf("输入整数集合A:\n");
    for (int i = 0; i < Anumber; i++) {
        scanf("%d", &A.data[i]);
    }
    //输入集合B的元素
    printf("输入整数集合B:\n");
    for (int i = 0; i < Bnumber; i++) {
        scanf("%d", &B.data[i]);
    }

    //求并集，存放于 A 并输出
    unionSet(&A, B);
    printf("A 和 B 的并集: ");
    printList(A);

    //重新初始化集合 A 和 B
    initList(&A);
    initList(&B);
    A.length = Anumber;
    B.length = Bnumber;

    // 求交集，存放于 A 并输出
    intersectSet(&A, B);
    printf("A 和 B 的交集: ");
    printList(A);

    return 0;
}

// 初始化顺序表
void initList(set* list) {
    list->length = 0;
}

// 输出顺序表
void printList(set list) {
    for (int i = 0; i < list.length; i++) {
        printf("%d ", list.data[i]);
    }
    printf("\n");
}

//求并集，结果存放于 A
void unionSet(set* A, set B) {
    for (int i = 0; i < B.length; i++) {
        int found = 0;
        //检查 B 的元素是否已经在 A 中
        for (int j = 0; j < A->length; j++) {
            if (B.data[i] == A->data[j]) {
                found = 1;
                break;
            }
        }
        //如果 B 中的元素不在 A 中，则将其插入 A 的末尾
        if (found == 0) {
            A->data[A->length] = B.data[i];
            A->length++;
        }
    }
}

//求交集，结果存放于 A
void intersectSet(set* A, set B) {
    int i = 0;
    while (i < A->length) {
        int found = 0;
        //检查 A 中的元素是否在 B 中
        for (int j = 0; j < B.length; j++) {
            if (A->data[i] == B.data[j]) {
                found = 1;
                break;
            }
        }
        //如果 A 中的元素不在 B 中，则从 A 中删除该元素
        if (found == 0) {
            //删除 A 中的第 i 个元素
            for (int k = i; k < A->length - 1; k++) {
                A->data[k] = A->data[k + 1];
            }
            A->length--;
        }
        else {
            i++;  //只有在元素不被删除的情况下才移动 i
        }
    }
}
```

运行结果：



心得体会：

在实现集合的并集与交集时，我对算法的设计有了更深入的认识。特别是在求并集时，逐个比较并添加不重复的元素，锻炼了我对循环和条件语句的运用能力。在求交集时，学会了在遍历中删除元素，这让我理解了动态数据结构的复杂性。另外，通过顺序表的实现，我对内存管理和数据存储的概念有了更清晰的认知。这次编程让我明白了算法效率与数据结构选择的重要性，获益颇丰。